

PODRĘCZNIK BEZPIECZEŃSTWA AGENTÓW AI (MANUAL)

MODUŁ 1: Bezpieczeństwo Agentów AI i Świadome Projektowanie Systemów

Rozdział 4. Katalog narzędzi operacyjnych agentów AI – co agent faktycznie „widzi” i może zrobić

Model językowy sam z siebie nie potrafi wykonywać działań poza generowaniem tekstu. Dopiero narzędzia rozszerzają jego możliwości o dostęp do danych, systemów, plików, Internetu czy interfejsów użytkownika. Jednakże z perspektywy bezpieczeństwa, każde dodatkowe narzędzie drastycznie zwiększa złożoność, koszt oraz powierzchnię potencjalnego ataku (*Attack Surface*).

4.1 URL CONTEXT – „ZWIADOWCA AI”

Definicja i Mechanizm Działania

URL Context pozwala agentowi analizować zawartość wskazanej strony internetowej. Działa jak zwiadowca wysłany na teren przeciwnika – pobiera dane zewnętrzne i wstrzykuje je jako dodatkowy kontekst do okna dialogowego LLM.

- **Proces:** Agent pobiera zawartość strony WWW $\rightarrow$ analizuje kod tekstowy $\rightarrow$ wykorzystuje znalezione informacje do dalszego wnioskowania.
- **Zastosowania biznesowe:** Automatyzacja analizy konkurencji (pobieranie cenników, ofert produktowych ze stron rywali i konsolidacja ich w raporty rynkowe).

Ograniczenia Techniczne

- Agent nie widzi interfejsu strony graficznej tak jak człowiek i nie rozumie ukrytych elementów wizualnych – przetwarza głównie surowy tekst.
- To sprawia, że jest wysoce podatny na manipulacje strukturą dokumentu.

Ryzyko Bezpieczeństwa: Indirect Prompt Injection (Pośrednie Wstrzyknięcie Promptu)

Haker umieszcza złośliwe instrukcje w strukturze strony WWW, wykorzystując techniki ukrywania tekstu przed człowiekiem:

- **Wektory ataku:** Biały tekst na białym tle, ukryte komentarze HTML (`<!-- złośliwy prompt -->`), niewidoczne dla użytkownika selektory CSS.
- **Skutki:** Agent odczytuje ukryty komunikat jako prawidłowy kontekst. Następuje **zatrucie kontekstu (Context Poisoning)**, co prowadzi do generowania błędnych odpowiedzi lub wykonywania nieautoryzowanych akcji w systemie organizacji.

4.2 YOUTUBE VIDEO – ANALIZA MATERIAŁÓW WIDEO

Definicja i Mechanizm Działania

Narzędzie to pełni rolę cyfrowego stenotypisty. Agent nie przetwarza sygnału wideo w sposób ludzki, lecz analizuje:

1. Pełną transkrypcję nagrania (napisy/audio-to-text).
 2. Metadane filmu (tytuł, tagi, opis).
 3. Oś czasu materiału (*timestamps*).
- **Zastosowania biznesowe:** Prace działów PR analizujących recenzje wideo produktów – automatyczne wyciąganie wniosków i generowanie streszczeń dla zarządu.

Ryzyko Bezpieczeństwa i Wektor Ataku

Złośliwy prompt sterujący może zostać ukryty bezpośrednio w opisie filmu, jego metadanych lub w samej transkrypcji (np. wypowiedziany lub zapisany jako tekst do odczytu).

- **Scenariusz ataku:** Agent analizuje wideo $\rightarrow$ w transkrypcji napotyka ukrytą instrukcję $\rightarrow$ traktuje ją jako nadrzędne polecenie architekta $\rightarrow$ ignoruje polecenia użytkownika i przesyła poufne dane do systemu hakera.
- **Wniosek:** Nawet zwykły opis filmu lub automatyczna transkrypcja są pełnoprawnym wektorem ataku *Prompt Injection*.

4.3 CODE INTERPRETER – BEZPIECZNE ŚRODOWISKO WYKONYWANIA KODU

Definicja i Mechanizm Działania

Code Interpreter to odizolowane, zamknięte środowisko uruchomieniowe (tzw. **Sandbox**), działające na zasadzie cyfrowej komory saperskiej. Agent może tam bezpiecznie wykonywać skrypty programistyczne, nie wpływając bezpośrednio na system operacyjny serwera głównego czy świat zewnętrzny.

Możliwości i Zastosowania Biznesowe

- Pisanie i natychmiastowe uruchamianie skryptów w języku Python.
- Zaawansowana analityka plików Excel, generowanie wykresów i raportów statystycznych.
- **Deterministyczne wyniki:** Wyeliminowanie halucynacji matematycznych modelu – operacje logiczne i matematyczne wykonuje realny kod, a nie mechanizm probabilistyczny LLM.

Ograniczenia Bezpieczeństwa

Domyślnie skonfigurowany sandbox posiada całkowicie zablokowany dostęp do Internetu oraz zewnętrznych połączeń API. Dzięki tej izolacji ryzyko wycieku przetwarzanych wewnątrz danych na serwery zewnętrzne jest minimalne.

4.4 COMPUTER USE – CYFROWY OPERATOR INTERFEJSU

Definicja i Mechanizm Działania

Computer Use umożliwia agentowi bezpośrednie sterowanie interfejsem graficznym (GUI) systemu operacyjnego w sposób emulujący ruchy człowieka.

- **Możliwości:** Ciągła analiza wizualna ekranu (zrzuty ekranu), poruszanie kursorem myszy, klikanie przycisków, wpisywanie tekstu w pola formularzy i obsługa aplikacji desktopowych.

Zastosowania Biznesowe

Rozwiązanie to jest niezastąpione w automatyzacji procesów manualnych (wprowadzanie danych, rutynowe czynności administracyjne), gdy organizacja operuje na systemach starszego typu (*Legacy Systems*), które nie posiadają interfejsów API.

Ryzyka Operacyjne i Bezpieczeństwa

Ponieważ agent działa bezpośrednio na żywym interfejsie użytkownika, błędy w interpretacji obrazu lub drobny dryf środowiska (np. przesunięcie okna aplikacji) mogą wywołać krytyczne skutki:

- Wykonanie nieprawidłowych kliknięć.
- Przypadkowe lub złośliwe usunięcie danych biznesowych.
- Wykonanie niepożądanych operacji systemowych paraliżujących procesy.
- **Wniosek:** *Computer Use* wymaga najbardziej rygorystycznej kontroli, twardych ograniczeń systemowych i ciągłego monitoringu (*Observability*).

4.5 COMPUTE OVERHEAD – UKRYTY KOSZT AGENTÓW

Na czym polega problem?

Każde podłączone do agenta narzędzie generuje stały koszt obliczeniowy i finansowy przy każdym zapytaniu do modelu, jeszcze zanim zostanie realnie użyte. Działa to jak taksówka z włączonym licznikiem stojąca na postoju.

Mechanizm Powstawania Kosztów

Aby model LLM wiedział, jakimi narzędziami dysponuje, każde z nich musi zostać szczegółowo opisane w jego prompcie systemowym za pomocą struktury definicji JSON (nazwa, opis działania, parametry wejściowe i wyjściowe). Model musi przetworzyć i przeanalizować całą listę tych definicji przy **każdym pojedynczym zapytaniu** użytkownika.

$$\text{Całkowity Koszt Promptu} = \text{Zapytanie Użytkownika} + \sum (\text{Definicje JSON Wszystkich Narzędzi})$$

Skutki Compute Overhead

- **Token Bloat:** Drastyczne, niepotrzebne zużycie tokenów wejściowych przez opisy nieużywanych narzędzi.
- Zwiększenie opóźnień (*Latency*) – wolniejsze odpowiedzi modelu.
- Wzrost kosztów utrzymania infrastruktury API.

Ryzyko Krytyczne: Infinite Looping (Nieskończona Pętla)

Sytuacja, w której agent błędnie interpretuje wynik działania narzędzia i wywołuje je ponownie w pętli zamkniętej:

$$\text{Wywołanie narzędzia} \rightarrow \text{Wynik} \rightarrow \text{Błędna interpretacja} \rightarrow \text{Ponowne wywołanie}$$

Skutkuje to lawinowym wzrostem wywołań API, przeciążeniem infrastruktury i gigantycznymi kosztami finansowymi wygenerowanymi w krótkim czasie.

4.6 TOOL OVERLOAD – MIT „SUPER-AGENTA”

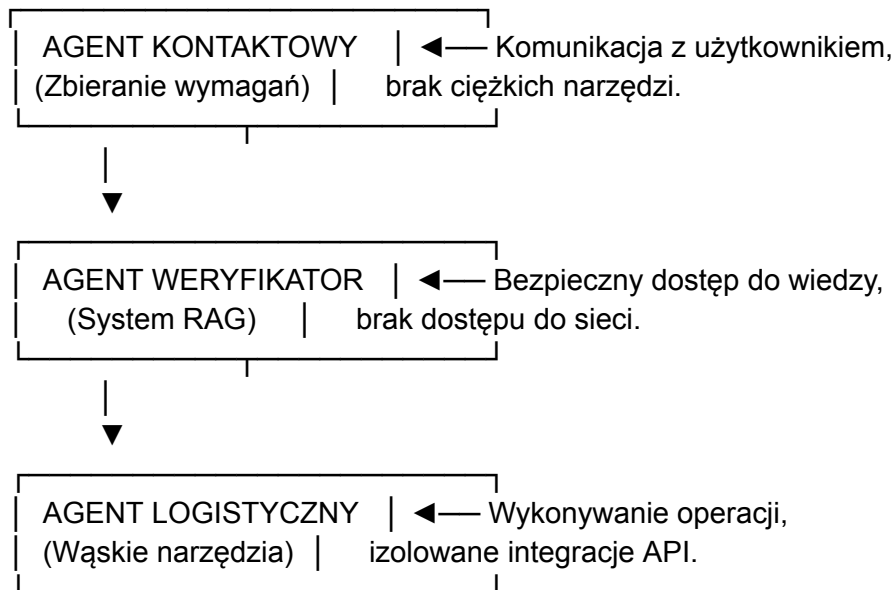
Problem Złożoności i Alignment Loss

Intuicyjne przekonanie deweloperów, że im więcej narzędzi posiada agent, tym jest on lepszy, jest błędem projektowym. Przy dziesiątkach integracji i skryptów agent zaczyna poświęcać większość zasobów kognitywnych na analizę własnych możliwości zamiast na rozwiązanie problemu. Doprowadza to do dwóch zjawisk:

- **Alignment Loss (Utrata zgodności celu):** Agent skupia się na zarządzaniu narzędziami i gubi główny cel biznesowy zadania.
- **Token Bloat:** Degradacja jakości odpowiedzi wynikająca z przeciążenia kontekstu opisami funkcji.

Dobra Praktyka Architekta AI

Należy projektować agentów wysoce wyspecjalizowanych o jasno zdefiniowanej roli, wyposażonych w maksymalnie **1–3 kluczowe narzędzia**. Złożone procesy biznesowe należy dzielić na systemy wieloagentowe (MAS):



4.7 BEZPIECZNA ARCHITEKTURA AGENTÓW AI

Współczesne projektowanie systemów autonomicznych opiera się na dwóch modelach kontroli nad uprawnieniami narzędzi:

A. Bezpieczeństwo Klasy Korporacyjnej (Microsoft Copilot)

Wdrożenie agenta w ekosystemie korporacyjnym wymusza automatyczne dziedziczenie zabezpieczeń infrastruktury IT:

- **RBAC (Role-Based Access Control):** Kontrola dostępu oparta na rolach w organizacji.
- **Security Trimming:** Dynamiczne filtrowanie danych – agent fizycznie "widzi" i przetwarza wyłącznie te zasoby i pliki, do których bezpośrednio uprawnienia posiada wywołujący go w danym momencie pracownik.

B. Pełna Kontrola Architektoniczna (GeneratorGPT / Własny AI Wrapper)

Zastosowanie własnej warstwy pośredniczącej (*AI Wrapper*) pomiędzy użytkownikiem a modelem LLM daje architektowi absolutną kontrolę nad bezpieczeństwem:

- Możliwość sztywnego definiowania uprawnień agentów i walidacji promptów.
- Izolacja modeli i precyzyjne filtrowanie narzędzi.

Fundament: Architektura Minimalnych Uprawnień (Principle of Least Privilege)

Agent powinien posiadać wyłącznie te uprawnienia i dostępy, które są absolutnie niezbędne do wykonania jego mikro-zadania. **Minimum dostępu = Maksimum bezpieczeństwa.**

- *Przykład:* Agent Audytor analizuje zagrożenia w izolacji sieciowej (brak dostępu do Internetu), a Agent Wykonawca realizuje operacje wyłącznie w oparciu o bezpieczną

bazę wiedzy RAG i odizolowane transakcje. Nie wolno dawać agentom pełnej, niekontrolowanej autonomii systemowej.

PODSUMOWANIE MATRYCY NARZĘDZIOWEJ (ŚCIAĞA OPERACYJNA)

Narzędzie operacyjne	Co faktycznie „widzi” agent?	Główne ryzyko (Red Team)	Zabezpieczenie (Blue Team)
URL Context	Surowy tekst ze stron WWW.	Indirect Prompt Injection (ukryty tekst).	Sanityzacja kodu HTML przed wstrzyknięciem do kontekstu.
YouTube Video	Transkrypcję, metadane, oś czasu.	Prompt Injection ukryty w napisach/opisie filmu.	Filtrowanie słów kluczowych w plikach transkrypcji.
Code Interpreter	Odizolowany system plików w Sandboxie.	Próby eskalacji uprawnień wewnątrz kontenera.	Całkowity brak dostępu do Internetu i zewnętrznych API.
Computer Use	Zrzuty ekranu (GUI).	Destrukcyjne operacje interfejsu (usunięcie danych).	Wdrożenie mechanizmu HITL (Human-in-the-Loop).
Wszystkie Narzędzia	Definicje parametrów JSON.	Infinite Looping, Token Bloat, Alignment Loss.	Redukcja liczby narzędzi do 1-3 na jednego agenta.